

Task 2: Regex-Based Normalization of Repeated Characters in Social Media Posts

Information Retrieval Project

Existing Tesla social-media dataset | Experimental subset: 100 posts

1. Introduction

Social-media text frequently contains deliberate character repetitions used to express emotion, emphasis, excitement, or informal pronunciation. Examples in the experimental data include *Goood*, *Brooooooooo*, *Begooo*, *Yasss*, *waaaaay*, *noooooothing*, and *Nyaaaaaaaaa....*. These forms can introduce noise for information-retrieval and natural-language-processing systems.

This task develops a regular-expression-based preprocessing pipeline to detect excessive alphabetic character repetition and normalize it while preserving important social-media elements such as URLs, mentions, and hashtags.

2. Objective

- Use another 100 posts from the existing project dataset.
- Identify repeated-character patterns occurring three or more times consecutively.
- Design a regular expression to detect excessive alphabetic repetition.
- Normalize repeated characters without deleting or altering complete URLs, mentions, or hashtags.
- Compare original and normalized posts and analyze successful corrections and errors.

3. Dataset

The existing Tesla social-media dataset was reused rather than collecting a new external dataset. Posts already used for Task 1 were excluded. Among the remaining data, 159 posts matched the repeated-alphabetic-character selection criterion. A sample of 100 posts was then selected using `random_state=42`. The resulting 100 posts form the Task 2 dataset.

Dataset property	Value
Source	Existing Tesla social-media dataset
Task 1 posts excluded	50
Matching posts available	159
Task 2 posts selected	100
Selection	Random sample; <code>random_state = 42</code>
Normalized output	<code>data/task2_normalized.csv</code>

4. Repeated-Character Pattern Analysis

The inspection stage used the pattern `( [A-Za-z] )\1{2,}`. It identifies an alphabetic character occurring at least three consecutive times. The uploaded inspection output shows both short expressive repetitions and extremely long repetitions.

Observed form	Example	Interpretation
Vowel repetition	Goood	Expressive spelling
Long repetition	Broooooo	Emphasis/excitement
Word-final repetition	Yasss	Informal emphasis
Consonant repetition	asssss	Expressive spelling
Extreme repetition	Nyaaaaaaaa...	Very strong emphasis
Uppercase repetition	NOOOOO...OT	Strong emphasis

5. Regex Design

Detection regex: `( [A-Za-z] )\1{2,}`

The first capturing group stores an alphabetic character. The backreference requires the same character to occur again, and the repetition threshold identifies sequences of at least three copies. This targets excessive alphabetic repetition rather than ordinary two-character sequences.

A lexical correction stage was also used because regex alone cannot determine the intended standard word. For example, reducing every repetition to one character would transform *goood* into *god*, whereas the intended normalization is *good*.

6. Protection of URLs, Mentions, and Hashtags

Before normalization, URLs, mentions, and hashtags are temporarily protected using a separate regular expression. They are replaced by placeholders, repeated-character normalization is applied to the remaining text, and the original protected strings are restored.

This prevents repetitions inside these structured social-media elements from being treated as ordinary spelling noise. For example, a mention such as *@Teslaaaaa* and a hashtag such as *#Teslaaaaa* are intended to remain exactly unchanged.

7. Normalization Examples

Original	Normalized	Assessment
Goood morning Twitter ...	Good morning Twitter ...	Successful
Broooooo congrats	Bro congrats	Successful
teslaaaaa	tesla	Successful
noooooothing	nothing	Successful
sweeeeet!	sweet!	Successful
Nyaaaaaaaa...	Nya...	Aggressive reduction
@Teslaaaaa / #Teslaaaaa	Unchanged	Protected

8. Experimental Results

- **100 posts** were processed in the Task 2 experimental subset.
- **159 posts** were available after applying the repeated-character selection criterion.

- The 100-post subset was selected with **random_state=42** after excluding Task 1 posts.
- The uploaded output contains both changed and unchanged posts.
- The output demonstrates recovery of conventional forms such as *good*, *nothing*, *sweet*, and *cool* from expressive variants.

Statistical reporting note: Exact aggregate values such as the number of changed posts and total characters removed should be calculated directly from the final CSV by the evaluation script. They are not invented here.

9. Error Analysis

Over-normalization: Not every three-or-more-character sequence is noise. The dataset contains legitimate-looking repetitions such as AAA and QQQ, as well as uppercase emphasis. A generic repetition rule can therefore modify valid abbreviations, symbols, or intentional emphasis.

Structured entities: The protection strategy is designed to keep URLs, mentions, and hashtags intact. However, the current implementation does not protect every possible social-media entity. The uploaded output shows examples where cashtags such as \$MOOO or \$QQQ can be altered by a generic alphabetic rule. This is a limitation to address in a more complete pipeline.

Lexical ambiguity: A repeated string may be slang, a username, a multilingual word, or a deliberate expressive form. Frequency-based correction can improve standard-word recovery but cannot guarantee the writer's intended word in every case.

Language variation: The selected posts include multilingual and code-mixed content. An English-only lexical frequency model is therefore less reliable for all posts.

10. Strengths

- Simple and computationally efficient regex-based detection.
- Uses real social-media examples from the project's existing dataset.
- Separates structured URLs, mentions, and hashtags before normalization.
- Can recover conventional forms better than a purely mechanical one-character reduction.
- Produces an original-versus-normalized dataset suitable for further evaluation.

11. Limitations

- Regex alone cannot infer the intended dictionary word.
- Repeated characters can carry meaningful emphasis and should not always be removed.
- Mixed-language posts reduce the reliability of English lexical correction.
- Cashtags and other structured entities need additional protection rules.
- Extreme normalization can remove expressive information.

12. Conclusion

The experiment demonstrates that regular expressions can provide an effective first stage for detecting repeated-character noise in informal social-media text. The 100-post dataset contains many forms of expressive repetition, ranging from short forms such as *Yasss* to extreme sequences such as *Nyaaaaaaaaa...* Lexical correction improves recovery of conventional words, while protection of URLs, mentions, and hashtags helps preserve important social-media structure. The error analysis also shows

that normalization must be conservative because repeated characters may be meaningful and may occur in legitimate identifiers or financial symbols.

13. AI Usage Statement

AI assistance was used for coding guidance, regex formulation, preprocessing-pipeline design, and report structuring. The dataset was taken from the existing project data, and the 100-post Task 2 subset and normalization outputs were generated in the student's local environment. Experimental observations, examples, error analysis, and conclusions were based on the uploaded experimental files. AI suggestions were critically reviewed, particularly regarding the limitations of regex-only normalization and the need to preserve structured social-media elements.

14. Project Outputs

- data/task2_posts.csv — 100-post Task 2 dataset.
- results/task2_repeated_patterns.txt — repeated-character inspection output.
- data/task2_normalized.csv — original and normalized posts.
- src/task2_select_posts.py — dataset selection script.
- src/task2_inspect.py — repeated-character inspection script.
- src/task2_normalize.py — normalization pipeline.